

Investigación sobre Alternativas y Estrategias de Almacenamiento para IntellOps

1. Contexto y alcance

IntellOps se despliega en un servidor propio del laboratorio GIDAS, sin equipo dedicado de administración de bases de datos. El uso inicial estimado es de 7 a 8 becarios por día utilizando las herramientas web monitoreadas. Este informe evalúa, para ese contexto puntual, qué estrategia de almacenamiento conviene para el MVP y qué cambios tendría sentido evaluar si el proyecto crece más allá de esta prueba inicial.

Esta versión del informe actualiza la decisión de motor: tras revisarlo con el equipo, se optó por PostgreSQL desde el MVP, en lugar de SQLite (decisión original de la v1.0 de este documento). El motivo central fue evitar migrar de motor más adelante y no encontrarse con las limitaciones de SQLite recién cuando el proyecto ya esté en uso real.

Aclaracion importante:

Para esta etapa de MVP, el sistema se valida utilizando datos mockeados que simulan el comportamiento de un servidor real, y no tráfico proveniente de usuarios reales del laboratorio. Esto implica que el volumen de datos manejado durante las pruebas no refleja necesariamente el uso real que tendría IntellOps una vez desplegado en producción. Se recomienda que, a medida que el proyecto sea retomado por nuevos integrantes y se incorpore tráfico real de usuarios del laboratorio, se revisen los supuestos de volumen aquí planteados.

2. Volumen Esperado

Tomando el ejemplo de referencia (una sesión de usuario genera del orden de 50 registros de métricas: tiempos de carga, clicks, llamadas a la API), el volumen diario estimado es:

Variable	Estimacion
Usuarios/día	7-8 becarios
Registros por sesion	~50
Filas/dias estimadas	~350-400
Filas/anos estimadas	~130.000-150.000

Con este volumen, ningún motor relacional estándar tiene problemas de rendimiento — la elección entre SQLite y PostgreSQL para el MVP no se decide por volumen, sino por lo que se explica en el punto 3

3. Porque PostgreSQL?

- **Un solo escritor a la vez.** Incluso en modo WAL, SQLite serializa las escrituras. Si el proyecto suma más de un servicio del GIDAS enviando métricas en paralelo (algo que ya está previsto como escenario de crecimiento), esto se vuelve una limitación real antes de lo esperado.
- **Sin gestión de usuarios ni permisos.** SQLite no tiene roles ni control de acceso a nivel de motor — cualquier proceso con acceso al archivo tiene acceso total. PostgreSQL permite separar permisos (por ejemplo, que el agente RUM solo pueda insertar, y el dashboard solo pueda leer).
- **Sin acceso por red.** SQLite es un archivo local; si en el futuro otro servicio del laboratorio necesita consultar la base directamente (no solo a través de la API), PostgreSQL ya lo resuelve de forma nativa.
- **Camino directo a TimescaleDB.** Como se detalla en la sección 8, TimescaleDB es una extensión de PostgreSQL. Empezar directamente en Postgres evita el paso intermedio de migrar de motor (SQLite → Postgres) el día que se necesite escalar — solo haría falta habilitar la extensión.
- **Ecosistema de herramientas.** Backups, réplicas, monitoreo (pg_stat_statements), y extensiones (JSONB+GIN, TimescaleDB, PostGIS si hiciera falta más adelante) están mucho más maduras en PostgreSQL.

En resumen: para el volumen actual, SQLite rendía bien, pero PostgreSQL da margen operativo para crecer sin rediseñar nada, a cambio de un costo de infraestructura menor (ver 3.2).

3.2 Costo de usar PostgreSQL con este volumen

Sumar PostgreSQL implica un proceso más corriendo en el servidor del laboratorio (a diferencia de SQLite, que no necesita proceso propio). Según los números ya presupuestados en containers.md para el resto del stack, un contenedor de PostgreSQL para este volumen ronda los ~150-250MB de RAM — un costo bajo comparado con otros componentes del stack (por ejemplo el LLM Server, ~600MB). No representa una complejidad operativa significativa: alcanza con Docker/docker-compose para levantarlo, que es el mismo mecanismo que ya usa el resto del proyecto.

4 Flexibilidad de esquema: JSONB + índice GIN

Con PostgreSQL ya como motor, se propuso usar JSONB para campos flexibles (por ejemplo un campo tags con {"browser": "Chrome", "btn_clicked": "submit_login"}), sin necesidad del equivalente JSON1 que se plantea para SQLite.

El patrón recomendado:

1. Columna tags de tipo JSONB (no JSON plano — JSONB guarda el contenido ya parseado en binario, lo que lo hace más rápido de consultar y es lo que permite indexarlo).
2. Consultas con operadores nativos: tags->>'browser', tags @> '{"browser": "Chrome"}'.
3. Si una consulta sobre un campo específico se vuelve frecuente, crear un índice GIN sobre la columna (o sobre una expresión puntual dentro del JSON). Esto le da a Postgres rendimiento de índice para consultas sobre el contenido del JSON, evitando escanear cada fila.

Esto permite agregar nuevos campos de metadata sin migrar el esquema ni reescribir tablas existentes.

5 Retención de datos

Se recomienda un mecanismo simple de purga desde el MVP: una tarea programada que elimine registros de RUM_METRIC más allá de una ventana definida (por ejemplo 90 días, valor que ya usa containers.md como default). En PostgreSQL esto se puede resolver con un cron del sistema operativo (igual que en SQLite) o con la extensión pg_cron, que permite programar la tarea dentro de la misma base de datos.

6 vale la pena usar TimescaleDB?

TimescaleDB es una extensión de PostgreSQL diseñada para datos de series temporales: particiona automáticamente las tablas por tiempo ("hypertables") y optimiza las consultas de agregación sobre rangos de fechas.

Con el volumen actual del MVP, no todavía — el particionado automático (hypertables) y las agregaciones continuas de TimescaleDB están pensadas para volúmenes de series temporales bastante mayores a las ~130.000-150.000 filas/año estimadas. Adoptarla ahora sería complejidad sin beneficio medible.

Al haber elegido PostgreSQL desde el MVP, sumar TimescaleDB el día de mañana es simplemente **habilitar la extensión** (CREATE EXTENSION timescaledb) y convertir RUM_METRIC en una hypertable — no implica migrar de motor. Es la principal ventaja concreta de esta decisión frente a haber seguido en SQLite.

7 separación física hot/cold

Si JS_EXCEPTION (texto pesado, consultado poco) empieza a representar una porción significativa del almacenamiento, se puede evaluar moverla a un tablespace separado del que usa RUM_METRIC, para no afectar la velocidad de lectura de esta última.

8. Conclusión

El equipo decidió usar PostgreSQL desde el MVP, La razón no es de volumen —con 7-8 becarios/día cualquiera de los dos motores responde bien— sino operativa: PostgreSQL evita las limitaciones de escritura concurrente y gestión de accesos de SQLite, y deja el camino a TimescaleDB reducido a habilitar una extensión en lugar de migrar de motor.

Esta investigación es un punto de partida y está sujeta a revisión a medida que el equipo confirme el comportamiento real del sistema en producción.